

TradeVision AI

Full-Stack Trading Strategy App

Complete Build Documentation

Stack	React + FastAPI (Python)
Platform	Web App (PWA-ready)
Contest	Builderr.ai Trading Agent Challenge
Dev	Solo — Meet
Version	1.0.0 — June 2026

This document covers everything needed to build TradeVision AI from scratch — architecture, backend logic, frontend screens, API contracts, finance concepts, deployment, and Builderr.ai submission guide.

01 — Platform Decision: Web App vs Mobile App

Why Web App wins for this project

You asked what is better — mobile app or something else. Here is the honest answer for TradeVision AI specifically:

Factor	Web App (React)	Mobile App (React Native)	Winner
Charts & data viz	Recharts, D3 — perfect	Limited, needs native libs	Web
Builderr.ai submit	Direct — agent.py + web URL	Not applicable	Web
Build speed (solo)	Fast — one codebase	Slower — iOS + Android	Web
Finance dashboards	Industry standard	Uncommon, complex	Web
PWA (install on phone)	YES — works offline too	Native only	Web
Python backend	Connects via REST easily	Same but more setup	Web
Portfolio demo	Strong for judges	Impressive but overkill	Web

Verdict

Build a React Web App with PWA support. It installs on mobile like an app, works in the browser, connects to Python backend, and submits cleanly to Builderr.ai. Best of all worlds for a solo dev.

Stack Overview

Frontend	React 18 + Vite + Recharts + TailwindCSS
Backend	Python 3.11 + FastAPI + Uvicorn
Data	yfinance (free) + sample OHLCV CSV fallback
Strategy	Custom decide() function — pure Python
Backtest	Custom engine — simulates 1 year of daily trades
Metrics	Calmar ratio, Sharpe, Max Drawdown, Win Rate
Submit	agent.py → push to GitHub → email builderr.ai
Deploy	Railway (backend) + Vercel (frontend) — both free tier

Project Folder Structure

```
tradevision/
├── backend/
│   ├── main.py           # FastAPI app entry point
│   ├── agent.py          # decide() – submitted to Builderr.ai
│   ├── backtest.py       # Backtesting engine
│   ├── metrics.py        # Calmar, Sharpe, drawdown calc
│   ├── data.py           # yfinance price fetcher
│   └── strategies/
│       ├── momentum.py   # Dual momentum strategy
│       ├── sma_crossover.py # SMA 50/200 crossover
│       └── vol_target.py  # Volatility targeting
│   └── requirements.txt
├── frontend/
│   ├── src/
│   │   ├── App.jsx
│   │   └── pages/
│   │       ├── Dashboard.jsx # Main portfolio view
│   │       ├── Backtest.jsx  # Run + view backtest
│   │       ├── Signals.jsx   # Buy/sell signals
│   │       └── Strategy.jsx   # Config + params
│   └── components/
│       ├── EquityCurve.jsx # Recharts line chart
│       └── DrawdownChart.jsx
```

```
■ ■ ■ ■ TradeTable.jsx
■ ■ ■ ■ MetricCard.jsx
■ ■ ■ ■ api/
■ ■ ■ ■ client.js      # Axios API calls
■ ■ ■ ■ package.json
■ ■ ■ ■ README.md
```

Every term explained simply

Calmar Ratio	The main score for Builderr.ai. Formula: Annualized Return / Max Drawdown. Example: +20% return with only -5% worst dip = Calmar of 4.0. Higher = better. A Calmar above 2.0 is excellent.
Max Drawdown	The biggest percentage drop from a peak to a trough. Example: Portfolio goes 100 → 130 → 90. Drawdown = $(130-90)/130 = 30.7\%$. Builderr.ai disqualifies bots with >50% drawdown.
Sharpe Ratio	Return per unit of risk. Formula: $(\text{Return} - \text{Risk Free Rate}) / \text{Std Dev of Returns}$. Above 1.0 is good. Above 2.0 is excellent.
Momentum	Stocks that went up recently tend to keep going up. Strategy: rank stocks by 3-month or 12-month return, buy top performers.
SMA (Simple Moving Average)	Average closing price over N days. SMA-200 = average of last 200 days. If price is above SMA-200, market is in uptrend. If below, go to cash.
Risk-Off Switch	A rule that moves everything to cash when market looks dangerous. Most common: if SPY < its 200-day SMA, sell all positions and hold cash.
Volatility Targeting	Size positions based on how much a stock moves. If a stock is volatile, buy less. If calm, buy more. Keeps your overall portfolio risk constant.
Leverage Cap (1.5x)	Builderr.ai rule: total exposure cannot exceed 1.5x your equity. If you have \$100k, max positions = \$150k. Leveraged ETFs (TQQQ) count 3x.
Calmar vs Sharpe	Builderr.ai ranks by Calmar. Sharpe penalizes all volatility (good + bad). Calmar only penalizes drawdown. Build for Calmar = protect against big drops.
Long Only	Builderr.ai v0 = no short selling. You can only buy stocks, not bet against them. Going to cash (0 positions) is the 'safe mode'.

requirements.txt

```
fastapi==0.111.0
uvicorn[standard]==0.29.0
yfinance==0.2.40
pandas==2.2.2
numpy==1.26.4
python-dotenv==1.0.1
```

agent.py — The Core decide() Function

This is the file submitted to Builderr.ai. It contains your strategy logic. The strategy below is Dual Momentum with SMA risk-off — one of the strongest simple strategies for Calmar ratio.

```
import pandas as pd
import numpy as np

def decide(market_state, portfolio_state, cash) -> list[dict]:
    orders = []

    # --- Risk-off switch ---
    # If SPY is below its 200-day SMA, go fully to cash
    spyBars = market_state.get('SPY', [])
    if len(spyBars) < 200:
        return [] # Not enough data yet

    spy_closes = [b['close'] for b in spyBars]
    spy_sma200 = np.mean(spy_closes[-200:])
    spy_price = spy_closes[-1]

    if spy_price < spy_sma200:
        # Market is in downtrend – sell everything, go to cash
        positions = portfolio_state.get('positions', [])
        for pos in positions:
            orders.append({
                'ticker': pos['ticker'],
                'side': 'sell',
                'quantity': pos['quantity']
            })
        return orders

    # --- Momentum ranking ---
```

```

# Rank ETFs by 3-month return, buy top 3
universe = ['SPY', 'QQQ', 'IWM', 'XLK', 'XLF', 'XLE', 'XLV', 'GLD']
momentum = {}
for ticker in universe:
    bars = market_state.get(ticker, [])
    if len(bars) < 63: # ~3 months
        continue
    closes = [b['close'] for b in bars]
    ret_3m = (closes[-1] - closes[-63]) / closes[-63]
    momentum[ticker] = ret_3m

if not momentum:
    return []

# Sort by momentum, pick top 3
ranked = sorted(momentum.items(), key=lambda x: x[1], reverse=True)
top3 = [t for t, r in ranked[:3] if r > 0] # Only positive momentum

if not top3:
    return [] # All negative momentum – stay in cash

# --- Position sizing (equal weight, max 30% each) ---
equity = cash + sum(
    pos['quantity'] * portfolio_state['last_prices'].get(pos['ticker'], 0)
    for pos in portfolio_state.get('positions', [])
)
target_pct = min(0.30, 1.0 / len(top3))
target_val = equity * target_pct

# Sell positions no longer in top3
for pos in portfolio_state.get('positions', []):
    if pos['ticker'] not in top3:
        orders.append({'ticker': pos['ticker'],
                       'side': 'sell', 'quantity': pos['quantity']})

# Buy top3 if not already held
held = {p['ticker'] for p in portfolio_state.get('positions', [])}
for ticker in top3:
    if ticker not in held:
        price = portfolio_state['last_prices'].get(ticker, 1)
        qty = int(target_val / price)
        if qty > 0 and cash >= qty * price:

```

```
        orders.append({'ticker': ticker, 'side': 'buy', 'quantity': qty})

        cash -= qty * price

    return orders
```

main.py — FastAPI Server

```
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware
from pydantic import BaseModel
from backtest import run_backtest
from data import fetch_prices
from metrics import calc_metrics

app = FastAPI(title='TradeVision API')

app.add_middleware(CORSMiddleware,
                   allow_origins=['*'], allow_methods=['*'], allow_headers=['*'])

class BacktestRequest(BaseModel):
    strategy: str = 'momentum'
    tickers: list[str] = ['SPY', 'QQQ', 'IWM']
    start_date: str = '2023-01-01'
    end_date: str = '2024-01-01'
    initial_cash: float = 100000

@app.get('/health')
def health(): return {'status': 'ok'}

@app.post('/backtest')
def backtest(req: BacktestRequest):
    prices = fetch_prices(req.tickers, req.start_date, req.end_date)
    result = run_backtest(prices, req.initial_cash, req.strategy)
    metrics = calc_metrics(result['equity_curve'])
    return {'*result', 'metrics': metrics}

@app.get('/prices/{ticker}')
def prices(ticker: str, period: str = '1y'):
    return fetch_prices([ticker], period=period)

if __name__ == '__main__':
    import uvicorn

    uvicorn.run(app, host='0.0.0.0', port=8000)
```


metrics.py — Calmar, Sharpe, Drawdown

```
import numpy as np
import pandas as pd

def calc_metrics(equity_curve: list[float]) -> dict:
    eq = np.array(equity_curve)
    returns = np.diff(eq) / eq[:-1]

    # Annualized return
    total_return = (eq[-1] - eq[0]) / eq[0]
    n_years = len(eq) / 252
    ann_return = (1 + total_return) ** (1 / n_years) - 1

    # Max drawdown
    peak = np.maximum.accumulate(eq)
    drawdown = (eq - peak) / peak
    max_dd = abs(drawdown.min())

    # Calmar ratio
    calmar = ann_return / max_dd if max_dd > 0 else 0

    # Sharpe ratio (assume 0% risk-free)
    ann_vol = returns.std() * np.sqrt(252)
    sharpe = ann_return / ann_vol if ann_vol > 0 else 0

    # Win rate
    wins = (returns > 0).sum()
    win_rate = wins / len(returns) if len(returns) > 0 else 0

    return {
        'total_return_pct': round(total_return * 100, 2),
        'ann_return_pct': round(ann_return * 100, 2),
        'max_drawdown_pct': round(max_dd * 100, 2),
        'calmar_ratio': round(calmar, 3),
        'sharpe_ratio': round(sharpe, 3),
        'win_rate_pct': round(win_rate * 100, 2),
    }
```

data.py — Price Fetcher

```
import yfinance as yf
import pandas as pd

def fetch_prices(tickers: list, start: str = None,
```

```

        end: str = None, period: str = '1y') -> dict:

result = {}
for ticker in tickers:
    try:
        if start and end:
            df = yf.download(ticker, start=start, end=end,
                             auto_adjust=True, progress=False)

        else:
            df = yf.download(ticker, period=period,
                             auto_adjust=True, progress=False)

        if df.empty:
            continue

        bars = []
        for ts, row in df.iterrows():
            bars.append({
                'ts':      str(ts.date()),
                'open':    round(float(row['Open']), 2),
                'high':    round(float(row['High']), 2),
                'low':     round(float(row['Low']), 2),
                'close':   round(float(row['Close']),2),
                'volume':  int(row['Volume']),
            })

        result[ticker] = bars
    except Exception as e:
        result[ticker] = {'error': str(e)}

return result

```

05 — API Endpoints Contract

Method	Endpoint	Request Body	Response	Used by
GET	/health	—	{ status: ok }	Health check
POST	/backtest	strategy, tickers, dates, cash	equity_curve, trades, metrics	Backtest page
GET	/prices/{ticker}	period (query param)	list of OHLCV bars	Chart page
GET	/strategies	—	list of strategy names + params	Config page
POST	/simulate/live	strategy params	today's signal + order	Dashboard

Sample /backtest Response

```
{
  "equity_curve": [100000, 101200, 99800, 103400, ...],
  "trades": [
    { "date": "2023-03-15", "ticker": "QQQ",
      "side": "buy", "qty": 10, "price": 318.5 },
    ...
  ],
  "metrics": {
    "total_return_pct": 18.4,
    "ann_return_pct": 19.1,
    "max_drawdown_pct": 7.2,
    "calmar_ratio": 2.65,
    "sharpe_ratio": 1.83,
    "win_rate_pct": 56.3
  }
}
```

Dashboard.jsx

Main screen

- Live equity value (big number top)
- Equity curve line chart (Recharts)
- Today's signal — BUY / SELL / HOLD badge
- Top 3 positions with current P&L;
- Quick metrics row: Calmar, Drawdown, Win Rate

Backtest.jsx

Run simulations

- Strategy selector dropdown
- Date range picker (start / end)
- Run Backtest button → POST /backtest
- Equity curve chart + drawdown chart
- Metrics cards grid
- Trades table with sortable columns

Strategy.jsx

Configure strategy

- Strategy picker (Momentum / SMA / Vol Target)
- Parameter sliders (lookback period, top-N, etc.)
- Risk-off toggle (enable/disable SMA filter)
- Universe selector (which tickers to trade)
- Save config button

Signals.jsx

Signal viewer

- Today's signals for all tickers
- Signal history table (last 30 days)
- Color coded: green=buy, red=sell, gray=hold
- Reason column explaining each signal

Key Components

```
// EquityCurve.jsx — Recharts line chart

import { LineChart, Line, XAxis, YAxis, Tooltip,
  ResponsiveContainer, ReferenceLine } from 'recharts'

export function EquityCurve({ data }) {
  return (
    <ResponsiveContainer width='100%' height={280}>
```

```
<LineChart data={data}>
  <XAxis dataKey='date' tick={{ fontSize: 11 }} />
  <YAxis tickFormatter={v => '$' + (v/1000).toFixed(0) + 'k'} />
  <Tooltip formatter={v => ['$' + v.toLocaleString(), 'Equity']} />
  <ReferenceLine y={100000} stroke='#94A3B8' strokeDasharray='4 4' />
  <Line type='monotone' dataKey='equity'
    stroke='#6366F1' strokeWidth={2} dot={false} />
</LineChart>
</ResponsiveContainer>
)
}
```

MetricCard.jsx — Stats display

```
export function MetricCard({ label, value, color = '#6366F1', sub }) {
  return (
    <div className='bg-slate-800 rounded-xl p-4 border border-slate-700'>
      <p className='text-slate-400 text-xs uppercase tracking-wide'>{label}</p>
      <p className='text-2xl font-bold mt-1' style={{ color }}>{value}</p>
      {sub && <p className='text-slate-500 text-xs mt-1'>{sub}</p>}
    </div>
  )
}

// Usage:
// <MetricCard label='Calmar Ratio' value='2.65' color='#10B981' sub='Target > 2.0' />
// <MetricCard label='Max Drawdown' value='-7.2%' color='#F87171' />
```

api/client.js — All API calls in one place

```
import axios from 'axios'

const API = axios.create({ baseURL: import.meta.env.VITE_API_URL
                               || 'http://localhost:8000' })

export const runBacktest = (params) =>
  API.post('/backtest', params).then(r => r.data)

export const getPrices = (ticker, period = '1y') =>
  API.get(`/prices/${ticker}`, { params: { period } }).then(r => r.data)

export const getLiveSignal = (params) =>
  API.post('/simulate/live', params).then(r => r.data)
```

Strategy 1: Dual Momentum (Recommended)

Rank a universe of ETFs by 3-month or 12-month return. Buy the top performers. Add an absolute momentum check — only buy if return > 0 (positive momentum). Switch to cash if SPY is below SMA-200. This strategy has historically delivered strong Calmar ratios by avoiding big drawdowns.

Lookback	63 days (3 months) or 252 days (12 months)
Universe	SPY, QQQ, IWM, GLD, TLT, XLK, XLF, XLE
Hold	Top 1-3 ETFs with positive momentum
Risk-off	SPY below SMA-200 → go to cash
Strength	Simple, low turnover, survives market crashes

Strategy 2: SMA Crossover

Buy when SMA-50 crosses above SMA-200 (golden cross). Sell when SMA-50 crosses below SMA-200 (death cross). Simple, well-known, but signals are slow — catches big trends.

Fast SMA	50-day moving average
Slow SMA	200-day moving average
Signal	Buy on golden cross, sell on death cross
Strength	Very simple, few trades, good for trending markets
Weakness	Whipsaws in sideways markets

Strategy 3: Volatility Targeting

Size each position inversely to its recent volatility. If SPY 10-day volatility is high, reduce exposure. If low, increase. Targets a fixed portfolio volatility (e.g. 10% annualized). This is how most professional risk managers think.

Target Vol	10% annualized (configurable)
Lookback	10-day rolling standard deviation of returns
Sizing	$\text{position_size} = \text{target_vol} / \text{current_vol}$
Max	Cap at 1.5x leverage (Builderr.ai rule)
Strength	Adapts to market conditions automatically

Color Palette (Dark Theme)

Name	Hex	Used for
Navy	#0F172A	Page background
Dark	#1E293B	Card backgrounds
Slate	#334155	Borders, dividers
Indigo	#6366F1	Primary accent, equity curve
Cyan	#22D3EE	Headers, highlights
Green	#10B981	Positive values, buy signals
Amber	#F59E0B	Warnings, labels
Red	#F87171	Negative values, sell signals
Light	#E2E8F0	Body text
Dim	#94A3B8	Muted text, subtitles

Typography

Font	Inter (Google Fonts) — clean, readable, finance-industry standard
Page title	24px, font-bold, text-white
Section H2	18px, font-semibold, text-cyan-400
Card label	11px, uppercase, tracking-wide, text-slate-400
Big number	32px, font-bold, colored by positive/negative
Table text	13px, text-slate-200
Code	Fira Code or Courier — monospace

Tailwind Config (tailwind.config.js)

```
module.exports = {
  content: ['./src/**/*.{js,jsx}'],
  theme: {
    extend: {
      colors: {
        brand: {
          indigo: '#6366F1',
          cyan: '#22D3EE',

```



```
    green: '#10B981',
    amber: '#F59E0B',
  },
  fontFamily: {
    sans: ['Inter', 'sans-serif'],
    mono: ['Fira Code', 'monospace'],
  },
}
}
```

Backend — Railway (Free)

```
# 1. Push backend/ to GitHub
cd backend && git init && git add . && git commit -m 'init'
git remote add origin https://github.com/yourusername/tradevision-backend
git push -u origin main

# 2. Go to railway.app → New Project → Deploy from GitHub

# 3. Set environment variable: PORT=8000

# 4. Railway auto-detects requirements.txt and deploys

# 5. Your API URL: https://tradevision-backend.up.railway.app
```

Frontend — Vercel (Free)

```
# 1. In frontend/, create .env file:
VITE_API_URL=https://tradevision-backend.up.railway.app

# 2. Push to GitHub, go to vercel.com → Import project

# 3. Set build command: npm run build

# 4. Set output directory: dist

# 5. Add environment variable VITE_API_URL

# 6. Deploy — your app URL: https://tradevision.vercel.app
```

Run Locally

```
# Backend

cd backend

pip install -r requirements.txt

python main.py

# → http://localhost:8000

# Frontend (new terminal)

cd frontend

npm install

npm run dev

# → http://localhost:5173
```

What to submit

File	agent.py — contains your decide() function
GitHub	Public repo with agent.py at root (or private with deploy key)
Email	submit@builderr.ai — send the GitHub repo link
Deadline	Round 1 runs June 2 – July 2, 2026 (30 days live trading)
Resubmit	You can push updates and resubmit any time during Round 1

Pre-submission checklist

- Run: python preview.py — confirms safety check will pass
- Run: python selfcheck.py — checks for secrets, syntax errors
- Verify no >50% drawdown on any test window
- Verify leverage never exceeds 1.5x (3x for TQQQ, 2x for QLD)
- Verify no single position >30% for more than 5 days
- Verify decide() runs in under 5 seconds
- Verify no lookahead bias (only use data up to current bar)
- Push agent.py to public GitHub repo
- Email link to submit@builderr.ai

Scoring reminder

Scored by Calmar Ratio = Annualized Return / Max Drawdown. +10% return with -2% worst dip (Calmar 5.0) beats +30% with -25% dip (Calmar 1.2). The risk-off switch in agent.py is your most powerful tool. Going to cash during downtrends protects your Calmar score.

11 — Build Order & Timeline

Day	Task	Files	Est. Time
1	Setup + backend skeleton	main.py, requirements.txt	2 hrs
1	Write agent.py (decide)	agent.py	1 hr
2	data.py + metrics.py	data.py, metrics.py	2 hrs
2	backtest.py engine	backtest.py	3 hrs
3	Test with preview.py	preview.py	1 hr
3	React app setup + routing	App.jsx, pages/	2 hrs
4	Dashboard + EquityCurve	Dashboard.jsx	3 hrs
5	Backtest page + charts	Backtest.jsx	3 hrs
6	Strategy + Signals pages	Strategy.jsx, Signals.jsx	3 hrs
7	Connect frontend to API	api/client.js	2 hrs
7	Deploy + submit	Railway, Vercel, GitHub	2 hrs

Total estimated: 7 days, ~24 hours of focused work. You can build faster since I will write most of the logic code for you.

12 — Extra Features to Make It Stand Out

AI Explain Button	Add a button next to each signal that calls Claude/GPT API to explain WHY the bot made that trade in plain English. 'QQQ was ranked #1 by 3-month momentum at +18.3%. SPY is above SMA-200 so market is in uptrend. Allocated 30%.'
Live Market Mode	Connect to a free real-time data source (Alpha Vantage free tier or Yahoo Finance) and show today's actual signal. Not live trading — just shows what the bot would do today.
Strategy Comparison	Run all 3 strategies simultaneously and show side-by-side Calmar, Sharpe, drawdown comparison. Helps pick the best strategy before submitting.
Dark/Light Theme Toggle	Simple toggle. Shows polish and attention to UX.
Export to CSV	Let users download the backtest trades as CSV. Simple but professional.
PWA Support	Add manifest.json and service worker. The app installs on phone like a native app. Impressive for a web app demo.
Shareable URL	Strategy config stored in URL params. Share your strategy config with one link. Example: /backtest?strategy=momentum&start;=2023-01-01

Ready to build. Say 'start backend' and I'll write the full Python code.

TradeVision AI — Built by Meet, June 2026